Sistemas Operativos 2026-2
Calendarización en Sistemas Operativos de Tiempo Real

October 18, 2026

Expediente	Nombre
219208106	Bórquez Guerrero Angel Fernando

---

1. Dados los procesos descritos en la siguiente tabla:

Tarea	Periodo (p)	Periodo de CPU o CPU Burst (t)	Deadline (d)
P1	50	35	50
P2	80	25	80

a) ¿Cuál es la utilización de CPU?

Para tareas periódicas, la utilización del CPU es $U = \sum_i \frac{t_i}{p_i}$:

$$U = \frac{35}{50} + \frac{25}{80} = 0.70 + 0.3125 = 1.0125 \;\Rightarrow\; U = 101.25\,\%$$

La utilización supera el 100 %, es decir, las tareas piden más tiempo de CPU del que el procesador puede ofrecer.

b) Demuestre si se pueden calendarizar con el algortimo RMS.

Asignación de prioridades (RMS): prioridad estática según el periodo; a menor periodo, mayor prioridad. Por lo tanto P_1 ($p = 50$) tiene mayor prioridad que P_2 ($p = 80$).

Prueba de utilización: para $n = 2$ tareas, RMS garantiza el cumplimiento de plazos si $U \leq 2\left(2^{1/2} - 1\right) \approx 0.8284$. Como $U = 1.0125 > 0.8284$ (y además $U > 1$), no se cumple la condición.

Demostración con la línea de tiempo:

- De $t = 0$ a $t = 35$ se ejecuta P_1 (termina antes de su plazo, $d = 50$).
- De $t = 35$ a $t = 50$ se ejecuta P_2: alcanza a usar solo 15 de sus 25 unidades.
- En $t = 50$ llega un nuevo trabajo de P_1, que tiene mayor prioridad y expulsa a P_2; P_1 se ejecuta de $t = 50$ a $t = 85$.
- En $t = 80$ vence el plazo de P_2, y solo ha recibido 15 de sus 25 unidades (le faltan 10). P_2 reanuda en $t = 85$ y termina en $t = 95 > 80$.

Conclusión: las tareas **no** se pueden calendarizar con RMS, pues P_2 incumple su plazo en $t = 80$.

c) Demuestre si se pueden calendarizar con el algoritmo EDFS

Condición de EDF: para tareas periódicas con plazo igual al periodo ($d = p$), EDF puede calendarizarlas si y solo si $U \leq 1$. Aquí $U = 1.0125 > 1$, por lo que no se cumple.

Demostración con el hiperperiodo: $\text{mcm}(50, 80) = 400$. En 400 unidades de tiempo, P_1 se ejecuta $400/50 = 8$ veces y P_2 se ejecuta $400/80 = 5$ veces, de modo que el trabajo total es

$$8(35) + 5(25) = 280 + 125 = 405 > 400.$$

El procesador solo dispone de 400 unidades, así que sobran 5 unidades de trabajo y, sin importar el algoritmo, algún plazo tiene que incumplirse.

Línea de tiempo: EDF atiende siempre primero la tarea con el plazo más cercano. Cumple todos los plazos durante el primer tramo, pero la sobrecarga se va acumulando hasta que el trabajo de P_1 liberado en $t = 350$ (plazo $d = 400$) termina hasta $t = 405$, es decir, 5 unidades tarde.

Conclusión: las tareas **no** se pueden calendarizar con EDF. Cabe notar que EDF resiste más que RMS (el primer incumplimiento ocurre en $t = 400$ y no en $t = 80$), pero ambos fallan porque $U > 1$.

2. Dados los procesos descritos en la siguiente tabla:

Tarea	Periodo (p)	Periodo de CPU o CPU Burst (t)	Deadline (d)
P1	50	15	50
P2	75	25	75
P3	100	20	100

a) ¿Cuál es la utilización de CPU?

$$U = \frac{15}{50} + \frac{25}{75} + \frac{20}{100} = 0.30 + 0.3333 + 0.20 = 0.8333 \;\Rightarrow\; U \approx 83.33\,\%$$

El CPU está ocupado el 83.33 % del tiempo y queda libre el 16.67 % restante.

b) Demuestre si se pueden calendarizar con el algortimo RMS.

Prioridades (RMS): P_1 ($p = 50$) $>$ P_2 ($p = 75$) $>$ P_3 ($p = 100$).

Prueba de utilización: para $n = 3$ tareas, el límite suficiente es $3\left(2^{1/3} - 1\right) \approx 0.7798$. Como $U = 0.8333 > 0.7798$, la prueba no garantiza ni descarta la calendarización, por lo que se requiere un análisis exacto.

Análisis exacto (tiempo de respuesta en el peor caso): $R_i = t_i + \sum_{j<i} \left\lceil \frac{R_i}{p_j} \right\rceil t_j$

- $R_1 = 15 \leq 50$ ✓
- $R_2 = 25 + \left\lceil \frac{R_2}{50} \right\rceil 15$: con $R_2 = 40$ se cumple ($\lceil 40/50 \rceil = 1 \Rightarrow 25 + 15 = 40$), así que $R_2 = 40 \leq 75$ ✓
- $R_3 = 20 + \left\lceil \frac{R_3}{50} \right\rceil 15 + \left\lceil \frac{R_3}{75} \right\rceil 25$: iterando desde $R_3 = 20$ se obtiene 60, luego 75, y se queda en 75 (con $R_3 = 75$: $20 + 2(15) + 1(25) = 75$). Por lo tanto $R_3 = 75 \leq 100$ ✓

Línea de tiempo (hiperperiodo $\text{mcm}(50, 75, 100) = 300$): el primer trabajo de P_3 es el más afectado, pues termina en $t = 75$ (plazo 100); los siguientes terminan en $t = 135$ (plazo 200) y $t = 275$ (plazo 300). Todos los trabajos de P_1 y P_2 también cumplen sus plazos.

Conclusión: las tareas **sí** se pueden calendarizar con RMS.

c) Demuestre si se pueden calendarizar con el algoritmo EDFS

Condición de EDF: con $d = p$, las tareas son calendarizables si y solo si $U \leq 1$. Aquí $U = 0.8333 \leq 1$, por lo que **sí se cumple**.

Verificación con el hiperperiodo (300): P_1 se ejecuta 6 veces ($6 \times 15 = 90$), P_2 4 veces ($4 \times 25 = 100$) y P_3 3 veces ($3 \times 20 = 60$): el trabajo total es $250 \leq 300$, y sobran 50 unidades en las que el CPU está libre.

Línea de tiempo: EDF atiende siempre la tarea con el plazo más cercano. Todos los trabajos terminan antes de su plazo; por ejemplo, el primer trabajo de P_3 termina en $t = 60$ (plazo 100), mejor que con RMS ($t = 75$).

Conclusión: las tareas **sí** se pueden calendarizar con EDF.